Expt. No. 5

Title: *Apply any one of stochastic gradient descent learning algorithms to learn the parameters of supervised single layer feed forward neural network*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 01/09/2025

Sign: _____

Experiment 5

Aim: Apply any one of the stochastic gradient descent algorithms to learn the parameters of a supervised single layer feed forward network.

Software: Google Collab

Prelab Question:

Explain the use of gradient descent algorithm in learning the parameters?

Gradient descent updates model parameters iteratively by moving them in the direction that minimizes the loss function.

Post Lab Question:

Explain various types of gradient descent algorithms?

Types of gradient descent include batch, stochastic, and mini-batch, differing in how many training samples are used per update.

Theory:

Gradient Descent is an iterative optimization process that searches for an objective function's optimum value (Minimum/Maximum). It is one of the most used methods for changing a model's parameters in order to reduce a cost function in machine learning projects.
The primary goal of gradient descent is to identify the model parameters that provide the maximum accuracy on both training and test datasets. In gradient descent, the gradient is a vector pointing in the general direction of the function's steepest rise at a particular point. The algorithm might gradually drop towards lower values of the function by moving in the opposite direction of the gradient, until reaching the minimum of the function.

Types of Gradient Descent:
Typically, there are three types of Gradient Descent:

1. Batch Gradient Descent
2. Stochastic Gradient Descent
3. Mini-batch Gradient Descent

Algorithm:

Stochastic Gradient Descent Algorithm

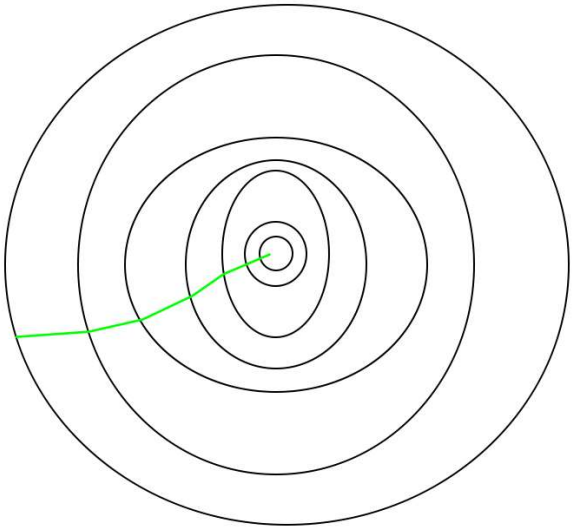
- Initialization: Randomly initialize the parameters of the model.
- Set Parameters: Determine the number of iterations and the learning rate (alpha) for updating the parameters.
- Stochastic Gradient Descent Loop: Repeat the following steps until the model converges or reaches the maximum number of iterations:
    - Shuffle the training dataset to introduce randomness.
    - Iterate over each training example (or a small batch) in the shuffled order.
    - Compute the gradient of the cost function with respect to the model parameters using the current training
      example (or batch).
    - Update the model parameters by taking a step in the direction of the negative gradient, scaled by the learning rate.

- Evaluate the convergence criteria, such as the difference in the cost function between iterations of the gradient.

- Return Optimized Parameters: Once the convergence criteria are met or the maximum number of iterations is reached, return the optimized model parameters.

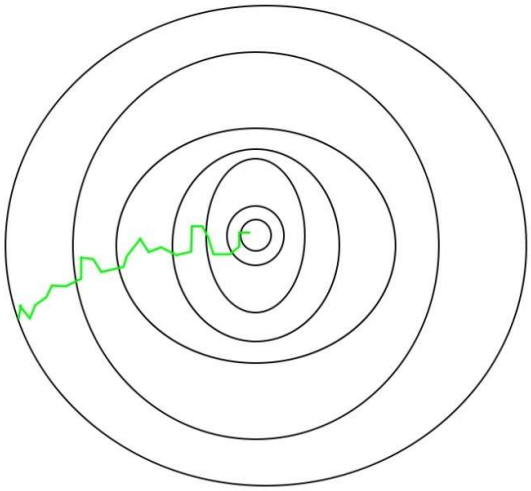
In SGD, since only one sample from the dataset is chosen at random for each iteration, the path taken by the algorithm to reach the minima is usually noisier than your typical Gradient Descent algorithm. But that doesn't matter all that much because the path taken by the algorithm does not matter, as long as we reach the minimum and with a significantly shorter training time.

The path taken by Batch Gradient Descent is shown below:



Batch gradient optimization path

A path taken by Stochastic Gradient Descent looks as follows –



stochastic gradient optimization path

One thing to be noted is that, as SGD is generally noisier than typical Gradient Descent, it usually took a higher number of iterations to reach the minima, because of the randomness in its descent. Even though it requires a higher number of iterations to reach the minima than typical Gradient Descent, it is still computationally much less expensive than typical Gradient Descent. Hence, in most scenarios, SGD is preferred over Batch Gradient Descent for optimizing a learning algorithm.

Program & Output

```
import numpy as np

class SGD:
    def __init__(self, lr=0.01, epochs=50, batch_size=32, tol=1e-3):
        self.learning_rate = lr
        self.epochs = epochs
        self.batch_size = batch_size
        self.tolerance = tol
        self.weights = None
        self.bias = None

    def predict(self, X):
        return np.dot(X, self.weights) + self.bias

    def mean_squared_error(self, y_true, y_pred):
        return np.mean((y_true - y_pred) ** 2)

    def gradient(self, X_batch, y_batch):
        y_pred = self.predict(X_batch)
        error = y_pred - y_batch
        gradient_weights = np.dot(X_batch.T, error) / X_batch.shape[0]
        gradient_bias = np.mean(error)
        return gradient_weights, gradient_bias

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.random.randn(n_features)
        self.bias = np.random.randn()

        for epoch in range(self.epochs):
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            for i in range(0, n_samples, self.batch_size):
                X_batch = X_shuffled[i:i+self.batch_size]
                y_batch = y_shuffled[i:i+self.batch_size]

                gradient_weights, gradient_bias = self.gradient(X_batch, y_batch)
                self.weights -= self.learning_rate * gradient_weights
                self.bias -= self.learning_rate * gradient_bias

            if epoch % 5 == 0:
                y_pred = self.predict(X)
```

```
            loss = self.mean_squared_error(y, y_pred)
            print(f"Epoch {epoch}: Loss {loss}")

        if np.linalg.norm(gradient_weights) < self.tolerance:
            print("Convergence reached.")
            break

    return self.weights, self.bias
```

Implementation

```
# Create random dataset with 100 rows and 5 columns
X = np.random.randn(100, 5)

# create corresponding target value by adding random noise in the dataset
y = np.dot(X, np.array([1, 2, 3, 4, 5])) + np.random.randn(100) * 0.1

# Create an instance of the SGD class
model = SGD(lr=0.01, epochs=300,
            batch_size=32, tol=1e-3)
w,b=model.fit(X,y)
# Predict using predict method from model
y_pred = w*X+b
```

OUTPUT:

```
Epoch 10: Loss 36.83902102412165
Epoch 15: Loss 24.319899960263573
Epoch 20: Loss 16.282765092319295
Epoch 25: Loss 10.321864503922669
Epoch 30: Loss 6.999428232341828
Epoch 35: Loss 4.827150728210429
Epoch 40: Loss 3.232828368260741
Epoch 45: Loss 2.1268789039079037
Epoch 50: Loss 1.4218184556972822
Epoch 55: Loss 0.9412865360555527
Epoch 60: Loss 0.6470548353796197
Epoch 65: Loss 0.449208824630044
Epoch 70: Loss 0.3095440661275809
Epoch 75: Loss 0.21041296761840184
Epoch 80: Loss 0.14413093126438314
Epoch 85: Loss 0.09590229924128563
Epoch 90: Loss 0.06832192284566814
Epoch 95: Loss 0.047815411982855105
Epoch 100: Loss 0.03433093863262353
```

```
Epoch 105: Loss 0.026396803814307 19
Epoch 110: Loss 0.020593096179001038
Epoch 115: Loss 0.016366146753255473
Epoch 120: Loss 0.013777339812725916
Epoch 125: Loss 0.012166408082821854
Epoch 130: Loss 0.011013998433223817
Epoch 135: Loss 0.010368027279322917
Epoch 140: Loss 0.009797246165275659
Epoch 145: Loss 0.009571011820295308
Epoch 150: Loss 0.009319837954281398
Epoch 155: Loss 0.009153042360763553
Epoch 160: Loss 0.009119868400032233
Epoch 165: Loss 0.009054382919628255
Epoch 170: Loss 0.009003554755430182
Epoch 175: Loss 0.008977847182312888
Epoch 180: Loss 0.008962163645710704
Epoch 185: Loss 0.008943006211655518
Epoch 190: Loss 0.008937114565367756
Epoch 195: Loss 0.008915529785911494
Epoch 200: Loss 0.008911057690807037
Epoch 205: Loss 0.00891031219151988
Epoch 210: Loss 0.008918415587164916
Epoch 215: Loss 0.008938118546777096
Epoch 220: Loss 0.008916932563959054
Epoch 225: Loss 0.008912583172415715
Epoch 230: Loss 0.008906888366635033
Epoch 235: Loss 0.008907746160737858
Epoch 240: Loss 0.00891786860580833
Epoch 245: Loss 0.008917162678634237
Epoch 250: Loss 0.008916430547949796
Epoch 255: Loss 0.008916129320380436
Epoch 260: Loss 0.008906114575013172
Epoch 265: Loss 0.008934911442453798
Epoch 270: Loss 0.008925626934384253
Epoch 275: Loss 0.008907678694029194
Epoch 280: Loss 0.00890465435804658
Epoch 285: Loss 0.00890956669364532
Epoch 290: Loss 0.008910838782186362
Epoch 295: Loss 0.0089094138027093
```

Conclusion:

Applying Stochastic Gradient Descent to a supervised single-layer feedforward neural network enables efficient optimization of weights by iteratively minimizing the error. This approach improves the model's learning speed and generalization on training data.

CO's Covered:

CO1. Implement basic neural network models.
CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance